

Document Tree

Gestion documentaire arborescente avec Django & DRF

Contexte

Tu intègres l'équipe backend d'une plateforme B2B SaaS dans l'industrie pharmaceutique française. La plateforme met en relation trois types d'acteurs :

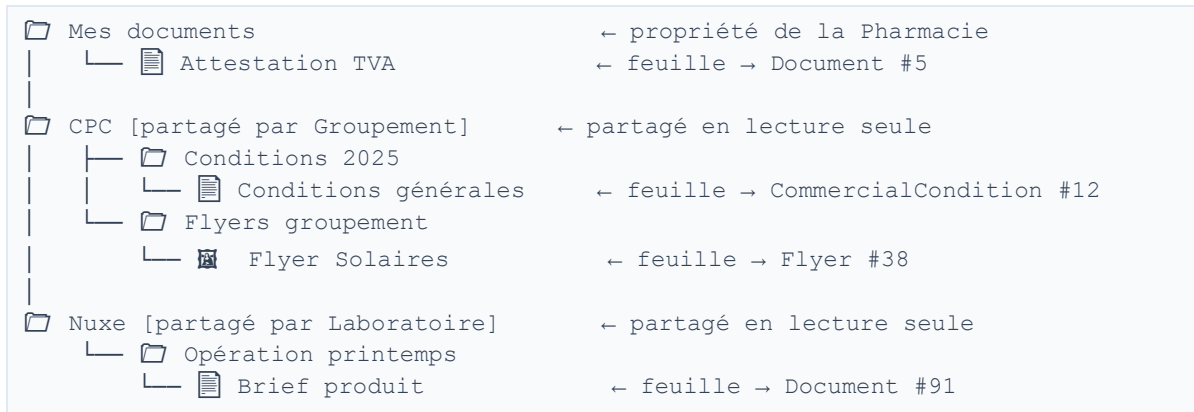
- Laboratoires — marques pharmaceutiques (ex : Nuxe, Bioderma)
- Groupements — réseaux/chaînes de pharmacies
- Pharmacies — officines individuelles, membres d'un groupement

Chaque entité produit et gère des documents de natures très diverses : documents génériques, flyers promotionnels, conditions commerciales... Ces objets sont aujourd'hui dispersés et difficiles à retrouver.

L'objectif : construire un système de gestion documentaire arborescente inter-entités. Chaque entité possède son propre arbre, mais peut partager des nœuds (dossiers ou feuilles) avec d'autres entités. Une Pharmacie voit dans son arbre ses propres documents ainsi que tout ce que son Groupement ou un Laboratoire lui a partagé.

Exemple : un Groupement crée un dossier « Conditions 2025 » contenant des conditions commerciales, et le partage avec toutes ses Pharmacies membres. Chaque Pharmacie voit ce dossier dans son arbre, en lecture seule.

Exemple d'arborescence vue par une Pharmacie membre du groupement CPC :



Objets métier fournis

Ces modèles existent déjà dans le projet — tu n'as pas à les modifier :

```

class Groupement(models.Model):
    name = models.CharField(max_length=255)

class Pharmacy(models.Model):
    name = models.CharField(max_length=255)
    groupement = models.ForeignKey(

```

```

        'Groupement', null=True, blank=True, on_delete=models.SET_NULL
    )

class Laboratory(models.Model):
    name = models.CharField(max_length=255)
    code = models.CharField(max_length=50, unique=True)

# --- Objets documentaires ---

class Document(models.Model):
    laboratory = models.ForeignKey('Laboratory', on_delete=models.CASCADE)
    name = models.CharField(max_length=255)
    file = models.FileField(upload_to='documents/')
    created_at = models.DateTimeField(auto_now_add=True)

class Flyer(models.Model):
    laboratory = models.ForeignKey('Laboratory', on_delete=models.CASCADE)
    title = models.CharField(max_length=255)
    image = models.ImageField(upload_to='flyers/')
    start_at = models.DateField()
    end_at = models.DateField()

class CommercialCondition(models.Model):
    laboratory = models.ForeignKey('Laboratory', on_delete=models.CASCADE)
    name = models.CharField(max_length=255)
    text = models.TextField()
    year = models.PositiveSmallIntegerField()

```

Phase 1 — Document d'architecture (1 à 2h)

Avant d'écrire une ligne de code, rédige un document qui répond aux questions suivantes.

Question 1 — Structure de l'arbre

Quelle stratégie de stockage retiens-tu pour l'arborescence ? (Adjacency List, MPTT, treebeard, Closure Table, Nested Sets...) Justifie ton choix en exposant les compromis lecture/écriture et les cas d'usage qui ont guidé ta décision.

Question 2 — Propriétaire polymorphe

Un nœud peut appartenir à un Laboratoire, un Groupement, ou une Pharmacie. Comment modélises-tu ce propriétaire sans FK en dur pour chaque type ? Est-ce le même mécanisme que pour le contenu des feuilles, ou un mécanisme différent ? Pourquoi ?

Question 3 — Modèle de partage

Comment modélises-tu le partage d'un nœud ? Un Groupement peut partager avec « toutes ses Pharmacies membres » ou avec « des Pharmacies spécifiques ». Un Laboratoire peut partager avec un Groupement entier ou des Pharmacies individuelles. Propose un modèle et explique ses limites.

Question 4 — Polymorphisme du contenu

Comment modélises-tu une feuille qui peut pointer vers un Document, un Flyer, ou une CommercialCondition — sans FK en dur pour chaque type ? Présente le modèle retenu et explique pourquoi.

Question 5 — Extensibilité

Dans 6 mois, on ajoute un nouveau type d'objet : Contract. Comment ton architecture absorbe-t-elle ce changement ? Une migration est-elle nécessaire ? Quelles parties du code sont touchées ?

Question 6 — Permissions et intégrité

Comment garantis-tu qu'une Pharmacie ne peut pas modifier un nœud partagé par son Groupement ? Et comment empêches-tu qu'un nœud soit partagé avec une Pharmacie qui n'appartient pas au bon Groupement ?

Question 7 — Vue agrégée

Quand une Pharmacie charge son arbre, elle doit voir ses propres nœuds ET les nœuds partagés avec elle (par son Groupement, par des Laboratoires). Comment construis-tu cette vue agrégée efficacement ? Quelles sont les implications en termes de performance ?

Question 8 — Design de l'API

Comment représentes-tu l'arbre en REST ? (liste plate + parent ID, JSON imbriqué, chemin filesystem...) Donne les signatures des 3–4 endpoints principaux. Comment l'API distingue-t-elle les nœuds possédés des nœuds partagés ?

Phase 2 — Proof of Concept (3 à 4h)

Implémente ton architecture.

- Modèles Django (document_tree/models.py) avec les relations de propriété et de partage
- Au moins 3 endpoints DRF fonctionnels :
 - Lister les enfants d'un nœud (navigation dans l'arbre)
 - Résoudre une feuille (retourner les données de l'objet sous-jacent)
 - Vue agrégée pour une entité donnée (ses nœuds + les nœuds partagés avec elle)
- Tests couvrant ces endpoints (cas nominal + au moins un cas d'erreur chacun)
- Endpoint de partage : partager un nœud avec une ou plusieurs entités
- Endpoint breadcrumb (chemin racine → nœud courant)
- Déplacement d'un nœud dans l'arbre
- Partage « toutes les pharmacies du groupement » (sans liste explicite)

Livrables attendus

1. Document d'architecture

Markdown, Google Doc, ou PDF.

2. Prototype développé

3. Tu utilises de l'AI ? tes conversations avec l'outils